



```
auto [is, js] = A.extents();
for(auto i : is)
  for(auto j : js)
    f(A(i, j));
```

Traditional loops

gpu::run

```
v = gpu::run(reduce(2), 3, A);
```

A[i][j]

Index access

A(i, j) A[i]



Try me online

"MULTI": multidimensional generic arrays in C++ (1D, 2D, 3D, ... ND)

```
#include <multi/array.hpp>
```

```
MPI_Send(message(G) ...)
```

MPI

cuFFT

```
multi::fft::forward(G);
```

Raw access (adaptors)

FFTW

cuBLAS

BLAS/LAPACK

```
w = multi::blas::gemv(G, v);
```

```
auto A = multi::array{
  {1, 2, 3},
  {4, 5, 6}};
```

Finally, better than Fortran/Numpy arrays in C++ for your CPU and GPU

A.begin()

A.end() Iterators

A.elements()

arxiv << A;

Boost

Standard Template Library

```
std::reduce(
  A.elements().begin(),
  A.elements().end())
```

CUDA Thrust

```
thrust::reduce(
  G.elements().begin(),
  G.elements().end())
```

```
multi::thrust::array G = A;
```

Clone me

